27 Week 09 058-307 FEB MON

M T W T F S S M T W T F S S
1 2 3 4 5 6 7 8 9 10 11 12
13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28

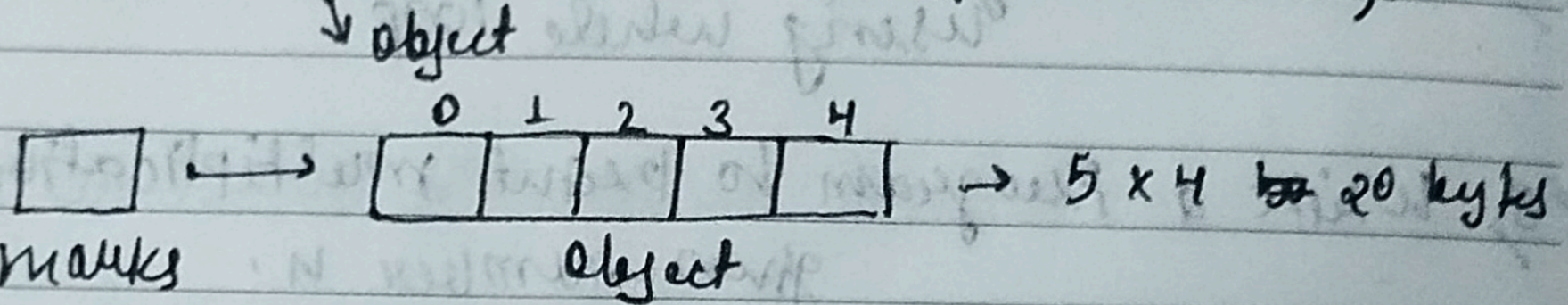
Chapter → 06

Introduction to Arrays

Array is a collection of similar types of data.

Use Case; Storing Marks of 5 Students

int [] marks = new int [5] ⇒ [dataType ArrName;].



Accessing Array Elements

Array elements can be accessed as follows:

marks[0] = 100
marks[1] = 70
⋮
marks[4] = 98

⇒ Note that index starts from 0.

So in a nut shell, this is how array works.

1. int [] marks; → Declaration!
   marks = new int[5]; → Memory Allocation!

2. int [] marks = new int[5]; → Declaration + Memory Allocation!

3. int [] marks = {100, 70, 80, 71, 98}; → Declare + Initialize!

2017

* Array indices starts from 0 and goes till (n-1) where n is the size of the array.

// Codes (Accessing Array Elements)

```
Public class Arrays
{
    public static void main(String[] args)
    {
        // Accessing Array Elements;
        // 1) Declaration and then Memory Allocations
        int[] marks;
        marks = new int[6];

        // 2) Declaration + Memory Allocations!
        int[] marks = new int[6];

        marks[0] = 96;
        marks[1] = 100;
        marks[2] = 70;
        marks[3] = 80;
        marks[4] = 75;
        System.out.println(marks[4]);

        // 3) Declare, Memory Allocation and Initialization
        int[] marks = {100, 70, 80, 71, 98};
        System.out.println(marks[4]);
    }
}
```

★ Array length

Arrays have a length property which gives the length of the array.

marks.length → gives 5 if marks is a reference to array with 5 elements.

• Displaying an Array

An array can be displayed using a for loop:

```
for (int i=0; i< marks.length; i++)
{
    System.out.println(marks[i]);
}
```

Quick Quiz

Write a Java Program to Print the elements of an array in reverse order.

```
public class ARRAYLENGTH
{
    Public static void main(String [] args)
    {
        int marks [] = { 10, 5, 7, 9, 3 };
        System.out.println( marks.length);
        for(int i = marks.length-1; i>=0; i-- )
        {
            System.out.println (marks[i]);
        }
    }
}
```

★ Multi-dimensional Arrays

Multidimensional arrays are array of arrays. Each Element of a M-D array is an array itself.
marks in the previous example was a 1-D array.

> 2-D Array

A 2-D array can be created as follows:

```
int [][] flats = new int [2][3]
```

↳ A 2-D array of 2 rows & 3 columns

we can add element to this array as follows

```
flats[0][0] = 100
flats[0][1] = 101
flats[0][2] = 102
   :
so on!
```

This 2-D array can be visualised as follows:

	[0] col 1	[1] col 2	[2] col 3
[0] Row 1	(0,0)	(0,1)	(0,2)
[1] Row 2	(1,0)	(1,1)	(1,2)

Similarly a 3-D array can be created as follows:

```
String [][][] arr = new String [2][3][4];
```

```
//code
public class MultiDimensionalArray
{
  Public static void main(String[] args)
  {
    System.out.println("Printing a 2-D array");
    int [][] apartment = new int[3][3];
    apartment[0][0] = 001;
    apartment[0][1] = 002;
    apartment[0][2] = 003;
    apartment[1][0] = 101;
    apartment[1][1] = 102;
    apartment[1][2] = 103;
    apartment[2][0] = 201;
    apartment[2][1] = 202;
    apartment[2][2] = 203;
    System.out.println(" Printing a 2-D Array using
                                    for loop");
    for(int i=0; i< apartment.length; i++)
    {
      for(int j=0; j< apartment[i].length; j++)
      {
        System.out.print(apartment[i][j]);
        System.out.print(" ");
      }
      System.out.println(" ");
    }
  }
}
```